

COSC363 Assignment 1 – Factory.cpp

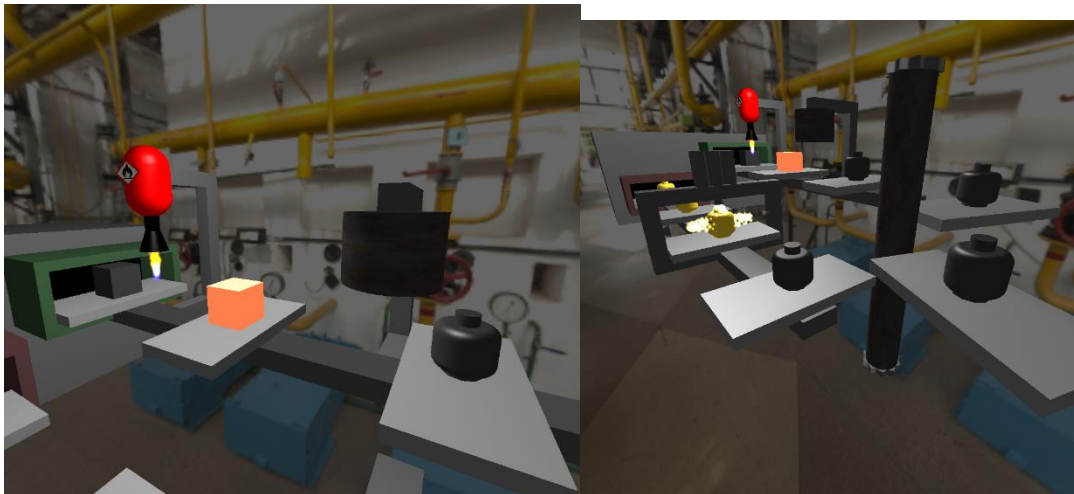
Luke Hyndman – 32098455 – 31/03/25

Scene Description

My scene features a small conveyor system that simulates an assembly line for a LEGO headpiece. Blocks of raw material exit a container, pass under a blowtorch for heating, and are then shaped by a drill/press. After rounding a corner and descending, the shaped heads are painted by nozzles before being returned to a different section of the container.

Key Features

Pictured below are the torch and press mechanism (left) and the downward spiral and paint station. (right)

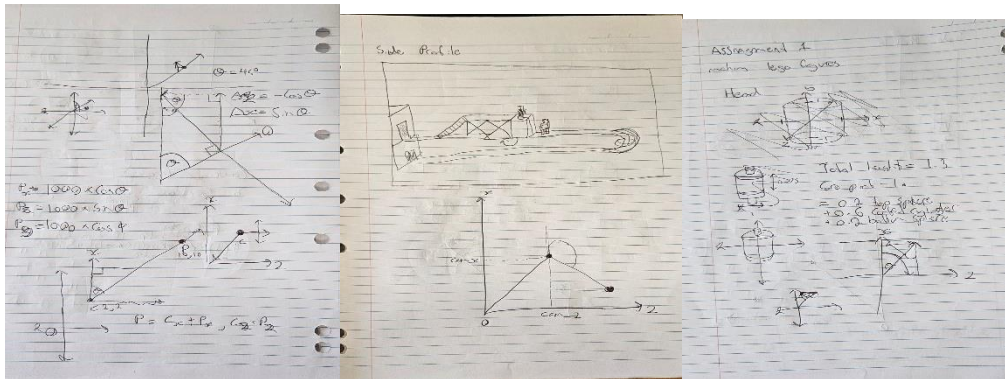


Extra Features

- **Moving conveyor:** The conveyor tiles that carry the heads move along a set of rails following a preprogrammed path
- **Non-linear conveyor:** Between the two straight sections of the conveyor, there is a section where it makes a half turn whilst spiraling downwards.
- **Sky sphere:** There is a sky sphere placed at $r=1000$
- **Custom-built sweep surface:** the pole that carries the tiles around the spiral has gears atop it that were made using quad and triangle strips. The surface was generated using a polar sine function in angular segments. (See equations section)
- **Surface shape generated by using a mathematical formula** (See above).
- **Particle systems:** There are two different particle systems, one for the torch and one for the paint. The torch particles have a complex color profile that changes with time to give the appearance of a red and blue flame. The particles are given randomized starting positions with velocities that are calculated to act counter to said positions therefore bringing all flame particles into a nice tip shape where they fade out. For the paint particles, constant start positions are assigned with randomized velocities to give the appearance of paint spraying from a nozzle. The paint particles are given lifetimes to disappear when they hit the Lego head.

Diagrams

Below are a few rough sketches used for camera positioning, conveyor design and animation calculations.



Equations

Many equations have been developed for various positions and timings throughout the scene. I decided to use time as my independent parameter for all animations.

Gear Design

For the sweep surface gear, A polar sinusoidal function was used where a radial distance from the centre would be returned as a function of angle.

```
float R(float theta, int N_TEETH, float TOOTH_SIZE, float GEAR_RADIUS) { // gear tooth function
    return sin(theta * N_TEETH / 2) * TOOTH_SIZE + GEAR_RADIUS;
}
```

The vertex normals for quads along the edge we found by using the theta derivative of R

```
void computeEdgeNormal(float theta, float normal[3], int N_TEETH, float TOOTH_SIZE) {
    float dRdtheta = cos(theta * N_TEETH / 2) * N_TEETH / 2 * TOOTH_SIZE;

    normal[0] = cos(theta) - dRdtheta * sin(theta);
    normal[1] = sin(theta) + dRdtheta * cos(theta);
    normal[2] = 0;

    float len = sqrt(normal[0]*normal[0] + normal[1]*normal[1]);
    normal[0] /= len;
    normal[1] /= len;
    normal[2] = 0;
}
```

The gear is then drawn as a sweep surface of quads with two flat faces of triangles forming the sides of the gear.

Animation Design

To animate the conveyor tiles and heads to move around the straights and curve in the belt, piecewise functions for x, y, z, and rotation about y were defined in terms of t. Note that time has been modulated to wrap around the total length of the animation cycle.

```
float tileX(float t){
    float t1 = fmod(t, TOTAL_TIME);
    float x;
    if (t1 >= TOP_START && t1 < SPIRAL_START) { // in the straight moving left to right
        x = X_SPEED * t1;
    } else if (t1 >= SPIRAL_START && t1 < BOTTOM_START) { // spiraling downwards
        float angle = (t1 - SPIRAL_START) * M_PI / (BOTTOM_START - SPIRAL_START);
        x = STRAIGHT_LENGTH + 0.5*SPIRAL_WIDTH*sin(angle);
    } else {
        x = STRAIGHT_LENGTH - X_SPEED*(t1-BOTTOM_START); // moving right to left
    }
    return x;
}
```

```
float tileY(float t){
    float t1 = fmod(t, TOTAL_TIME);
    float y;
    if (t1 >= TOP_START && t1 < SPIRAL_START) { // in the straight moving left to right
        y = 0;
    } else if (t1 >= SPIRAL_START && t1 < BOTTOM_START) { // spiraling downwards
        y = -1 * Y_SPEED * (t1 - SPIRAL_START);
    } else {
        y = -1 * DROP_HEIGHT; // moving right to left
    }
    return y;
}
```

To animate multiple heads on multiple tiles, a phase is added to t when it is passed to the positioning function.

```
void positionTile(float t, float phase) {
    glPushMatrix();
    float x = tileX(t + phase*TOTAL_TIME);
    float y = tileY(t + phase*TOTAL_TIME);
    float z = tileZ(t + phase*TOTAL_TIME);
    float rotation = tileAngle(t + phase*TOTAL_TIME);
    glTranslatef(x,y,z);
    glRotatef(rotation, 0, 1, 0);
    drawConveyorTile();
    glPopMatrix();
}

positionPress(t);
drawTrack(t);
for (float i = 0; i <= N_TILES; i++) {
    positionTile(t, i/N_TILES);
    positionHead(t, i/N_HEADS);
}
```

The timing for the various machines was done by calculating a period based on the total time and the number of heads on the belt and adjusting using an offset parameter.

Control Functions

There are 5 degrees of freedom for the camera. Space and Z keys move up and down in Y respectively. W, A, S, and D move the camera in the xy plane. The arrow keys can be used to pan left and right and to tilt the camera up and down. Q toggles the wireframe mode and R resets the camera to look at the scene from its initial position. These functions are controlled by special and keyboard callback functions and a helper function is used to calculate where the camera should be looking.

Build instructions

The program was developed on Visual Studio Codium for Linux machines and as such can be built and run using a GCC compiler through VS Codium. However, the program can also be run from the command line with the following steps.

- Open a terminal with CTRL+ALT+T.
- Ensure g++ is installed by entering g++ --version
 - If it is not installed use sudo apt update
 - sudo apt install g++
- Navigate to the folder containing the source code and compile using:

- `g++ -o Factory Factory.cpp legoman.cpp skybox.cpp conveyor.cpp particles.cpp gear.cpp -lGL -lGLU -lglut`
- Now move the executable into the build folder using:
`mv Factory build/`
- Now to move into the build folder using `cd build`
- Now run using: `./Factory`
- The program can be rerun using the above command as many times as you would like.
- If any changes are made, be sure to delete the old executable and recompile.

References

- The Skybox Texture was created by Oliksiy Yakovlyev and procured from Polyhaven:
https://polyhaven.com/a/boiler_room
- The Lego face texture was acquired from the Lego wiki
https://legouniverse.fandom.com/wiki/LEGO_Universe_Wiki?file=Jamesster_Face_Texture.png
- The metal texture used for the press and pole was acquired from OpenGameArt.org
<https://opengameart.org/content/rusted-metal-texture-pack> and was authored by user p0ss.
- The flame sign texture is from istockphoto.com
https://media.istockphoto.com/id/1066419604/vector/standard-pictogram-of-flammable-symbol-warning-sign-of-globally-harmonized-system.jpg?s=612x612&w=0&k=20&c=XdNDfwJX036QTWGm6_Of7USNZtY3ra3karWkjFlrziE=
- The BMP used for the particles was acquired from lab materials.

Use of Generative AI

GenAI (DeepSeek and GPT4) was used to explain the various kinds of GL_LIGHTING e.g. ambient, specular etc. It was also used to generate RGB values for the glowing red ingot. It was also used to understand how to calculate the vertex normals for the gear. It was also used for general debugging questions. AI was also used to proofread, and grammar-check this report